FALL 2026–2027 • THEORY + LAB

Digital Differential Analyzer (DDA) Line Drawing Algorithm

Complete step-by-step instructor handout | Derivation • Mathematics • Pixel tables • Pseudocode • OpenGL/FreeGLUT

Learning objectives

- Explain scan conversion/rasterization and why a continuous line must be approximated by discrete pixels.
- Use slope and the dominant direction to distinguish the m < 1, m > 1, and m = 1 DDA cases used in the provided lecture.
- Derive the incremental DDA update equations from y = mx + B rather than memorizing them.
- Solve the lecture example from (4,5) to (7,7), showing unrounded values and plotted integer pixels.
- Implement DDA in C++/OpenGL and explain the floating-point and round-off limitations stated in the lecture.

TEACHING NOTE *Teach DDA as a story: continuous line → integer pixel grid → slope → small repeated increments → rounding. Students understand the formulas much faster when they see why a fractional coordinate must be converted to a pixel.*

SOURCE ALIGNMENT The core derivation, slope cases, worked example, rounding rule, and disadvantages in this handout follow the provided "Chapter 2 (part_1)_Scan Conversation" lecture. The generalized implementation section is explicitly marked as an instructor extension.

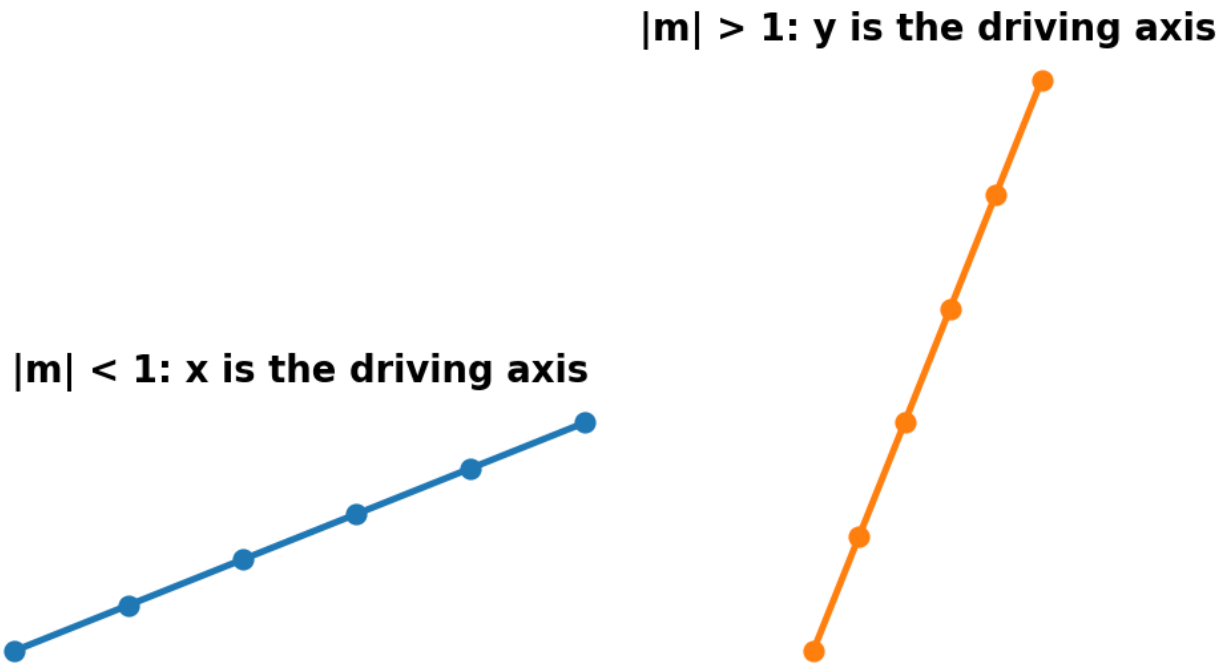
1 Where DDA fits in scan conversion

Rasterization converts vector-described shapes into raster pixels/dots. A line exists continuously in mathematics, but a display can illuminate only discrete pixel locations. A line drawing algorithm therefore chooses a sequence of integer pixels that approximates the ideal line.

$$P_0 = (x_0, y_0) \quad \rightarrow \quad P_n = (x_n, y_n)$$

The provided lecture treats DDA as an incremental algorithm: begin at the initial pixel, compute a small change, use it to obtain the next pixel, and continue until the destination is reached.

$$\Delta x = x_n - x_0 \qquad \Delta y = y_n - y_0 \qquad m = \Delta y / \Delta x$$



The slope tells us which coordinate should advance by one unit. For a shallow line, x is the natural driving coordinate. For a steep line, y is the natural driving coordinate.

2 The three source-aligned DDA slope cases

Case	Increment idea	Next-coordinate rule
m < 1	Advance x by 1	$x_{i+1} = x_i + 1$; $y_{i+1} = y_i + m$
m > 1	Advance y by 1	$y_{i+1} = y_i + 1$; $x_{i+1} = x_i + 1/m$
m = 1	Advance both by 1	$x_{i+1} = x_i + 1$; $y_{i+1} = y_i + 1$

TEACHING NOTE *In the source lecture these formulas are shown for the positive-slope teaching cases. Introduce these first. Handle reverse direction/negative slope later through the generalized implementation.*

3 Derivation for m < 1: why y increases by m

Start with the same line equation used in the lecture:

$$y = mx + B$$

Assume the current point on the ideal line is (x_i, y_i). Therefore:

$$y_i = mx_i + B$$

For a shallow line, advance x by one pixel. More generally write $x_{i+1} = x_i + \Delta x$. Then:

$$y_{i+1} = mx_{i+1} + B$$

$$y_{i+1} = m(x_i + \Delta x) + B$$

$$y_{i+1} = mx_i + B + m\Delta x$$

Because $mx_i + B = y_i$:

$$y_{i+1} = y_i + m\Delta x$$

For the lecture case $m < 1$, choose $\Delta x = 1$:

$$x_{i+1} = x_i + 1 \qquad y_{i+1} = y_i + m$$

The computed y value is generally fractional. The lecture then plots the nearest integer pixel.

$$\text{Plotted pixel} = (x_{i+1}, \text{round}(y_{i+1}))$$

For non-negative values, the lecture illustrates rounding using:

$$\text{round}(y) = [\, y + 0.5 \,] \quad \text{(lecture notation for non-negative } y\text{)}$$

TEACHING NOTE *Do not overwrite the running fractional y with its rounded display value. Keep the accumulated y for the next DDA step and round only the coordinate that is plotted. This preserves the intended incremental calculation.*

4 Derivation for m > 1 and the m = 1 special case

When the line is steep, increasing x by 1 can jump over rows. Instead, let y advance by one unit and determine the corresponding change in x.

$$m = \Delta y / \Delta x \quad \Rightarrow \quad 1/m = \Delta x / \Delta y$$

With $\Delta y = 1$ per step:

$$y_{i+1} = y_i + 1 \qquad x_{i+1} = x_i + 1/m$$

The lecture states the steep-line update in the same form:

$$\text{if } m > 1\text{:} \quad x_1 = \text{round}(x + 1/m), \quad y_1 = y + 1$$

For a 45° positive line, $m = 1$, so both coordinates increase together:

$$\text{if } m = 1\text{:} \quad x_1 = x + 1, \quad y_1 = y + 1$$

5 Worked example from the provided lecture: (4,5) → (7,7)

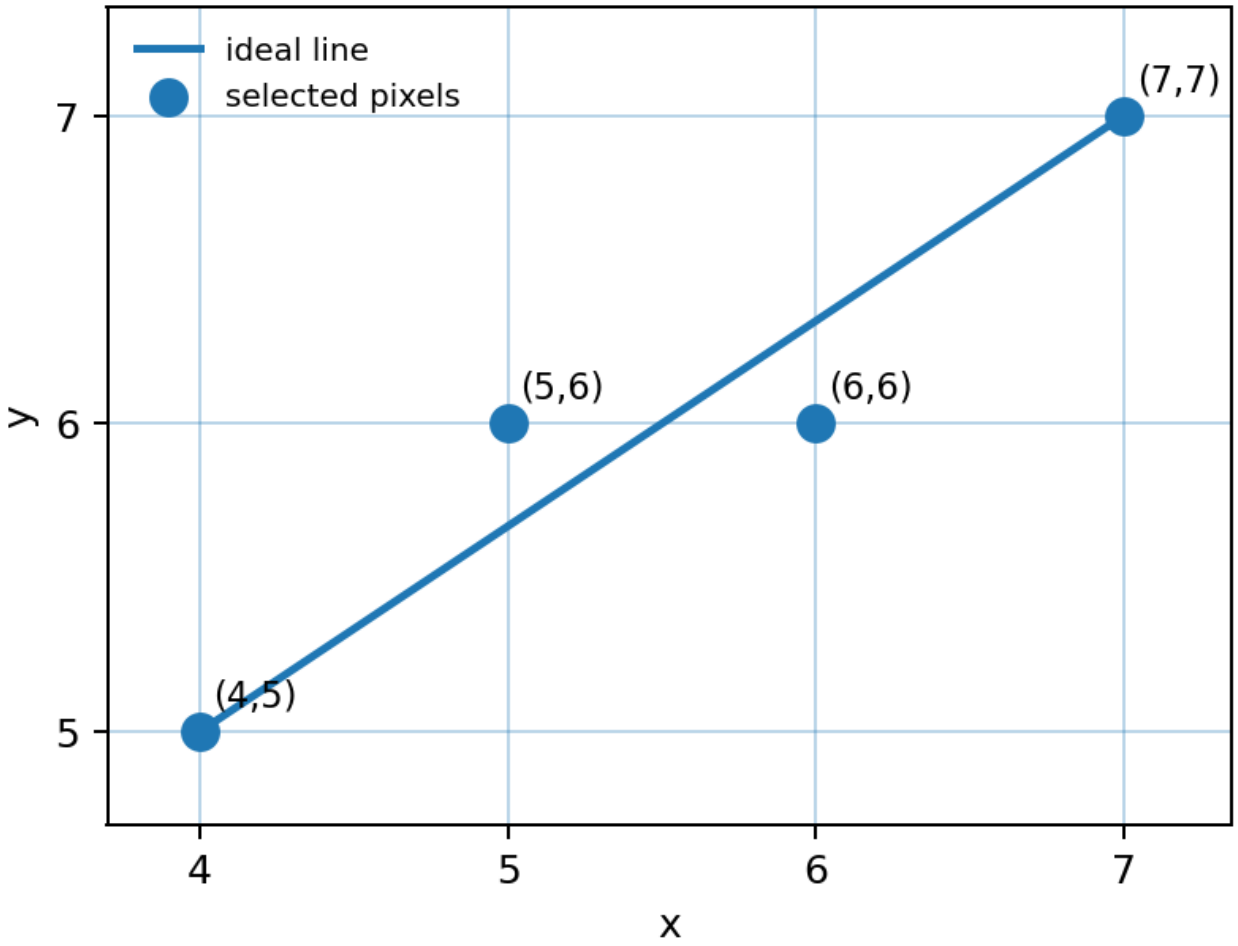
Initial pixel: (4,5). Destination pixel: (7,7).

$$\Delta x = 7 - 4 = 3 \qquad \Delta y = 7 - 5 = 2$$

$$m = \Delta y / \Delta x = 2/3 \approx 0.67$$

Since m < 1, increment x by 1 and add m to the running y value. The lecture uses the rounded slope 0.67 in its arithmetic.

Step	Running x	Running y (lecture arithmetic)	Pixel plotted
Start	4	5.00	(4,5)
1	5	5.00 + 0.67 = 5.67	(5,6)
2	6	5.67 + 0.67 = 6.34	(6,6)
3	7	6.34 + 0.67 = 7.01	(7,7)



Final pixel sequence:

$$(4,5),\ (5,6),\ (6,6),\ (7,7)$$

TEACHING NOTE *On the board, keep a separate "running y" column and "plotted y" column. This prevents the most common mistake: using the rounded plotted value as the next arithmetic value.*

6 Source-aligned DDA algorithm: board version

1. Read the initial pixel (x_0,y_0) and final pixel (x_n,y_n).
2. Compute $\Delta x = x_n - x_0$ and $\Delta y = y_n - y_0$.
3. Compute slope $m = \Delta y/\Delta x$ when $\Delta x \neq 0$.
4. If $m < 1$, repeatedly use $x \leftarrow x + 1$ and $y \leftarrow y + m$; plot (x, round(y)).
5. If $m > 1$, repeatedly use $y \leftarrow y + 1$ and $x \leftarrow x + 1/m$; plot (round(x), y).
6. If $m = 1$, increment both x and y by 1.
7. Continue until the destination coordinate is reached.

Slope condition	Driving coordinate	Increment	Pixel to plot
m < 1	x	x += 1; y += m	(x, round(y))
m > 1	y	y += 1; x += 1/m	(round(x), y)
m = 1	x and y	x += 1; y += 1	(x,y)

7 Instructor extension: generalized DDA used in code

SOURCE ALIGNMENT This section generalizes the classroom slope cases into a common programming form. It is useful for lab implementation but is not the step-by-step form shown in the source slides.

Use the larger absolute displacement as the number of increments:

$$\text{steps} = \max(|\Delta x|, |\Delta y|)$$

$$\text{xIncrement} = \Delta x / \text{steps} \qquad \text{yIncrement} = \Delta y / \text{steps}$$

Start at (x_0,y_0). At every iteration, plot the rounded current coordinate and then add both increments. This automatically makes the dominant coordinate change by one unit in magnitude and also handles reverse direction.

```
DDA(x1, y1, x2, y2)

    dx = x2 - x1
    dy = y2 - y1
    steps = max(abs(dx), abs(dy))

    xInc = dx / steps
    yInc = dy / steps

    x = x1
    y = y1

    plot(round(x), round(y))

    for i = 1 to steps
        x = x + xInc
        y = y + yInc
        plot(round(x), round(y))
```

TEACHING NOTE *For production-quality classroom code, protect the zero-length case where both endpoints are identical; otherwise steps = 0 would cause division by zero.*

8 C++ / OpenGL-FreeGLUT implementation

SOURCE ALIGNMENT The source lecture develops the mathematics, not a specific C++ listing. The following is an instructor/lab implementation of the generalized DDA formulation.

```
#include <GL/glut.h>
#include <cmath>
#include <algorithm>

void drawDDA(float x1, float y1, float x2, float y2)
{
    float dx = x2 - x1;
    float dy = y2 - y1;

    int steps = (int)std::max(std::fabs(dx), std::fabs(dy));

    // Same start and end point
    if (steps == 0)
    {
        glBegin(GL_POINTS);
        glVertex2i((int)std::round(x1), (int)std::round(y1));
        glEnd();
        return;
    }

    float xIncrement = dx / steps;
    float yIncrement = dy / steps;

    float x = x1;
    float y = y1;

    glBegin(GL_POINTS);

    for (int i = 0; i <= steps; i++)
    {
        glVertex2i((int)std::round(x), (int)std::round(y));
        x += xIncrement;
        y += yIncrement;
    }

    glEnd();
}
```

Line-by-line teaching points

- dx and dy measure horizontal and vertical displacement.
- steps chooses the dominant displacement, so the dominant coordinate advances by one pixel-sized interval per iteration.
- xIncrement and yIncrement are the repeated DDA changes.
- x and y remain floating-point running values.
- std::round() is used only when a point is sent to the raster grid.
- i <= steps includes both endpoints.

9 Disadvantages stated in the provided lecture

The source lecture highlights these limitations of DDA:

- Floating-point arithmetic is time consuming compared with an integer-only incremental approach.
- The algorithm is orientation dependent in its basic slope-case presentation.

- Endpoint accuracy can be poor in the basic formulation.
- Successive additions of a floating-point increment can accumulate round-off error; for long line segments the calculated pixel position may drift away from the true line path.

TEACHING NOTE *Use the disadvantages as the bridge to Bresenham: "DDA is simple, but it repeatedly carries fractional values and rounds them. Can we decide the next pixel with integer arithmetic instead?" That question naturally introduces the midpoint/Bresenham algorithm.*

10 Common student mistakes

Mistake	Correction
Using m = Δx/Δy	Slope is m = Δy/Δx.
Rounding the running y/x and then continuing from that rounded value	Keep the running fractional value; round only the plotted pixel.
Using the m < 1 update on a steep line	For m > 1, y is the driving coordinate and x changes by 1/m.
Forgetting the m = 1 special case	Both x and y increase by 1.
Dividing by zero for a vertical line or zero-length generalized case	Treat vertical/special cases explicitly or use a safe generalized implementation.

11 Suggested classroom sequence

1. Draw an ideal line across a large integer pixel grid and ask which pixels should be selected.
2. Review slope and show why shallow and steep lines have different natural driving coordinates.
3. Derive $y_{i+1} = y_i + m$ for m < 1 directly from y = mx + B.
4. Solve the (4,5) → (7,7) lecture example with a running-value table.
5. Show m > 1 and m = 1 after students understand the shallow case.
6. Demonstrate the generalized code and then ask students to change the endpoints.
7. Finish by discussing accumulated round-off error and motivate Bresenham.

Practice / viva questions

1. Define rasterization and explain why a line drawing algorithm is required.
2. What makes DDA an incremental algorithm?
3. Derive $y_{i+1} = y_i + m$ for m < 1.
4. Why is x the driving coordinate when m < 1?
5. Write the DDA updates for m > 1 and m = 1.
6. Using the lecture method, find the pixels from (4,5) to (7,7).
7. Why should the running fractional value not be replaced by the rounded pixel value?
8. State the disadvantages of DDA given in the lecture.
9. How does the generalized steps = max(|Δx|,|Δy|) form relate to the slope cases?

10. Why does DDA naturally motivate Bresenham's line algorithm?

Instructor answer guide

1. Rasterization maps continuous/vector geometry to discrete pixels; a line algorithm chooses the pixels that best approximate the ideal line.

2. DDA is incremental because each new coordinate is obtained by adding a fixed small increment to the previous coordinate.

3. From $y = mx + B$ and $x_{i+1} = x_i + 1$: $y_{i+1} = m(x_i+1)+B = y_i+m$.

4. For $m < 1$, x changes more rapidly across the grid, so advancing x by one avoids skipping columns.

5. For $m > 1$: y += 1 and x += 1/m. For $m = 1$: x += 1 and y += 1.

6. Lecture example pixels: (4,5), (5,6), (6,6), (7,7).

7. Rounding the running value changes the accumulated arithmetic; keep the fractional running value and round only the displayed pixel.

8. The lecture lists floating-point cost, orientation dependence/basic endpoint accuracy issues, and accumulated round-off drift.

9. steps = max(|Δx|,|Δy|) makes the dominant coordinate change by one unit in magnitude while the other changes proportionally.

10. DDA uses fractional increments and rounding; Bresenham is introduced to make pixel decisions with an incremental integer-style decision parameter.

12 One-page quick reference for teaching

$$\Delta x = x_2 - x_1 \qquad \Delta y = y_2 - y_1 \qquad m = \Delta y / \Delta x$$

Case	Formula
m < 1	$x_{ne}w = x + 1$; $y_{ne}w = y + m$; plot ($x_{ne}w$, round($y_{ne}w$))
m > 1	$y_{ne}w = y + 1$; $x_{ne}w = x + 1/m$; plot (round($x_{ne}w$), $y_{ne}w$)
m = 1	$x_{ne}w = x + 1$; $y_{ne}w = y + 1$

Lecture example: (4,5) → (7,7)

$$m = (7-5)/(7-4) = 2/3 \approx 0.67 < 1$$

$$(4,5) \rightarrow (5,6) \rightarrow (6,6) \rightarrow (7,7)$$

General coding form (instructor extension):

$$steps = \max(|\Delta x|, |\Delta y|)$$

$$xInc = \Delta x / steps \qquad yInc = \Delta y / steps$$

Key transition to Bresenham: DDA is easy to understand, but it carries floating-point increments and uses rounding; the lecture notes that repeated round-off can accumulate. Bresenham replaces that style of selection with an incremental decision parameter.

Source basis: provided "Chapter 2 (part_1)_Scan Conversation" lecture, DDA section (definition, derivation, slope cases, mathematics, and disadvantages).